LAPORAN PRAKTIKUM PEMROGRAMAN BACKEND LANJUT
MODUL 9 — TRANSACTIONS & CONCURRENCY
(EDISI PUZZLE)

Mata Kuliah	Praktikum Pemrograman Backend Lanjut (2 SKS)
Program Studi	D4 Teknik Informatika — Fakultas Vokasi, Universitas Airlangga
Nama Mahasiswa	Rozan Aiman Ramadani
NIM	434241069
Tautan Repository	https://github.com/RozanRamadani/tugas1_go (Folder: TM-9/latihan-fiber)

BAB I — LANDASAN TEORI & KONSEP INTI

1.1 Masalah Konkurensi pada Sistem KRS

Pada endpoint backend sederhana, operasi basis data umumnya hanya terdiri dari satu SQL statement yang secara otomatis bersifat atomik di PostgreSQL. Namun, pada alur pengisian Kartu Rencana Studi (KRS), sebuah request memerlukan banyak tahap: membaca sisa kuota, memvalidasi batas SKS mahasiswa, mencatat baris pendaftaran di tabel enrollments, dan menaikkan nilai enrolled_count pada tabel courses. Ketika puluhan mahasiswa menekan tombol ambil pada detik yang sama, pendekatan sekuensial naif mengalami kegagalan fatal berupa:

• Race Condition (Check-Then-Act): Mahasiswa sama-sama membaca sisa kursi masih ada, lalu keduanya sama-sama mendaftar sehingga kuota terlampaui.

• Lost Update: Dua penambahan jumlah peserta saling menimpa karena dihitung dari angka bacaan lama yang belum diperbarui.

1.2 Transaksi dan Tingkat Isolasi (READ COMMITTED)

Membungkus kode naif dengan blok transaksi (BEGIN ... COMMIT) semata TIDAK menyelesaikan masalah race condition. Pada tingkat isolasi bawaan PostgreSQL (READ COMMITTED), pernyataan SELECT biasa tidak mengunci baris data apa pun. Transaksi hanya menjamin sifat Atomicity (seluruh perubahan tersimpan atau seluruhnya dibatalkan), tetapi tidak otomatis membebaskan aplikasi dari bentrokan konkurensi.

1.3 Tiga Teknik Pengendalian Konkurensi

1. UPDATE Bersyarat yang Atomik: Menggabungkan pemeriksaan syarat kuota dan penulisan nilai dalam satu pernyataan SQL tunggal (UPDATE courses SET enrolled_count = enrolled_count + 1 WHERE id = $1 AND enrolled_count < quota).

2. Pessimistic Locking (SELECT ... FOR UPDATE): Mengunci baris data secara eksklusif sejak dibaca hingga transaksi selesai. Cocok untuk aturan batas 24 SKS yang datanya tersebar di banyak baris.

3. Optimistic Locking (Kolom version): Tidak mengunci data selama user/admin melihat form. Saat menyimpan, nomor versi diperiksa dan dinaikkan (UPDATE courses SET quota = $1, version = version + 1 WHERE id = $2 AND version = $3).

BAB II — D.1 BERITA ACARA PERBAIKAN (9 BUG PUZZLE)

Modul 9 Edisi Puzzle memuat 9 kesalahan sengaja pada Bagian B (Langkah Panduan), yang terdiri dari 3 kesalahan tingkat compiler dan 6 kesalahan tingkat perilaku/konkurensi. Berikut adalah berita acara perbaikan resmi sesuai format wajib modul:

No	Berkas dan Baris	Jenis	Gejala	Akar Masalah	Perbaikan dan Bukti
1	app/repository/ course_repository.go baris 33	Ditolak compiler	cannot use &coursePostgresRepository{…} as CourseRepository: wrong type for method WithTx (have WithTx(DBTX) *coursePostgresRepository, want WithTx(DBTX) CourseRepository)	Go mewajibkan return type method pada struct sama persis dengan interface. Method mengembalikan pointer konkrit *coursePostgresRepository alih-alih interface CourseRepository.	Ubah return type WithTx menjadi CourseRepository. Memenuhi C.7 butir 1.
2	tools/loadtest/ main.go baris 44	Ditolak compiler	invalid operation: cannot send to receive-only channel <-chan string out (variable of type <-chan string)	Parameter channel out dideklarasikan receive-only (<-chan string), sehingga operasi kirim out <- job(i) ditolak compiler Go.	Ubah tipe channel menjadi send-only (chan<- string). Memenuhi C.7 butir 1.
3	app/repository/ tx.go baris 34	Ditolak compiler	cannot use func(tx repository.DBTX) error as func(tx pgx.Tx) error in argument to s.tx.WithTx	Signature WithTx menerima func(tx pgx.Tx) error, bertentangan dengan komentarnya dan Clean Architecture yang mengabstraksikan database melalui DBTX.	Ubah parameter menjadi func(tx DBTX) error. Memenuhi C.7 butir 1.
4	app/repository/ tx.go baris 53	Lolos compiler, salah perilaku (Resource leak)	Koneksi pool habis setelah request gagal; koneksi berstatus idle in transaction menumpuk di pg_stat_activity hingga server macet.	Kode modul tidak memanggil defer tx.Rollback(ctx). Tanpa defer rollback, kegagalan transaksi membiarkan koneksi terbuka dan menyandera slot pool koneksi selamanya.	Tambahkan defer tx.Rollback(ctx) tepat setelah pool.Begin. Memenuhi C.5 dan C.7 butir 11.
5	app/repository/ course_repository.go baris 96	Lolos compiler, salah perilaku (Concurrency)	Pada mode credits, mahasiswa serentak dapat mendaftar hingga 27–36 SKS (melebihi batas maksimal 24 SKS).	Kueri LockStudent hanya SELECT id FROM users tanpa FOR UPDATE. Tanpa kunci pesimistik, pemeriksaan batas SKS bocor oleh request serentak dari mahasiswa yang sama.	Tambahkan FOR UPDATE pada kueri: SELECT id FROM users WHERE id = $1 FOR UPDATE. Memenuhi C.2 dan C.7 butir 3.
6	app/repository/ course_repository.go baris 117	Lolos compiler, salah perilaku (Constraint violation)	Mahasiswa ke-6 di kelas berkuota 5 menerima HTTP 500 INTERNAL_ERROR alih-alih 409 COURSE_FULL, memicu check constraint violation.	Klausa WHERE memakai enrolled_count <= quota. Saat count=5 dan quota=5, perbandingan 5 <= 5 bernilai true dan count dinaikkan menjadi 6 yang melanggar CHECK constraint.	Ubah klausul WHERE menjadi enrolled_count < quota. Memenuhi C.1, C.4, dan C.7 butir 5.
7	app/repository/ course_repository.go baris 145	Lolos compiler, salah perilaku (Data integrity)	Membatalkan kelas yang tidak pernah diambil menghasilkan 204 No Content, dan enrolled_count kelas itu berkurang secara keliru menjadi negatif.	DeleteEnrollment mengabaikan tag.RowsAffected(). Pada PostgreSQL, DELETE yang tidak menemukan row tetap berhasil dengan 0 baris (bukan error).	Tambahkan pengecekan: if tag.RowsAffected() == 0 { return ErrNotEnrolled }. Memenuhi C.4 dan C.7 butir 7.
8	app/service/	Lolos	Endpoint pindah kelas	Di dalam WithTx, kode	Ganti pemanggilan

	course_service.go baris 224	compiler, salah perilaku (Deadlock / Leak)	(/enrollments/swap) macet total selama 5 detik hingga dibatalkan oleh context deadline exceeded.	memanggil s.courses.DecrementEnrolled (lewat pool) alih-alih repo.DecrementEnrolled. Kueri dari pool terhalang oleh baris yang dikunci sendiri oleh transaksi tersebut.	menjadi repo.DecrementEnrolled agar berada di dalam transaksi. Memenuhi C.3, C.4, C.7 butir 4 & 9.
9	app/repository/ course_repository.go baris 182	Lolos compiler, salah perilaku (Optimistic Concurrency)	Admin mengirim version lama tetap berhasil mengubah kuota, dan nilai version pada response tidak pernah bertambah.	Kueri UPDATE hanya SET quota = $1 tanpa menaikkan nomor versi (version = version + 1), sehingga nomor versi di basis data tidak pernah berubah.	Ubah kueri menjadi SET quota = $1, version = version + 1. Memenuhi C.4 dan C.7 butir 10.

BAB III — D.2 IMPLEMENTASI PEMBIMBING TUGAS AKHIR (ENTITY STUDENTS)

3.1 Skema Basis Data & Constraint Relasi (007_supervisors.sql)

Kasus penetapan dosen pembimbing tugas akhir dibangun dengan mengintegrasikan entitas students yang telah dikembangkan sejak Modul 3. Aturan bahwa 'seorang mahasiswa hanya boleh memiliki satu pembimbing' dijamin secara mutlak di tingkat basis data melalui constraint UNIQUE pada kolom student_id di tabel supervisions:

```
CREATE TABLE IF NOT EXISTS supervisors (
    id SERIAL PRIMARY KEY,
    name VARCHAR(120) NOT NULL,
    quota INTEGER NOT NULL CHECK (quota > 0),
    supervised_count INTEGER NOT NULL DEFAULT 0,
    version INTEGER NOT NULL DEFAULT 1,
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    CONSTRAINT supervisors_count_within_quota
        CHECK (supervised_count >= 0 AND supervised_count <= quota)
);

CREATE TABLE IF NOT EXISTS supervisions (
    id BIGSERIAL PRIMARY KEY,
    student_id VARCHAR(50) NOT NULL UNIQUE REFERENCES students(id) ON DELETE CASCADE,
    supervisor_id INTEGER NOT NULL REFERENCES supervisors(id) ON DELETE CASCADE,
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
```

3.2 Potongan Logika Bisnis & Transaksi (supervisor_service.go)

1. Endpoint POST /supervisors/:id/assign:

Mengunci baris mahasiswa dengan SELECT id FROM students WHERE id = $1 FOR UPDATE, lalu mengambil kuota pembimbing secara atomik dengan UPDATE bersyarat WHERE supervised_count < quota, dan mencatat pendaftaran di supervisions.

2. Endpoint POST /supervisions/switch:

Memastikan atomicity penggantian pembimbing. Menghindari deadlock dengan mengunci kedua pembimbing secara terurut (SELECT id FROM supervisors WHERE id = ANY($1) ORDER BY id FOR UPDATE), lalu memotong kuota pembimbing lama dan menaikkan kuota pembimbing baru.

3. Endpoint PATCH /supervisors/:id:

Menerapkan optimistic locking kolom version (UPDATE supervisors SET quota = $1, version = version + 1 WHERE id = $2 AND version = $3).

BAB IV — D.3 BUKTI PENGUJIAN BEBAN & PERCOBAAN PEMBANDING

4.1 Uji Beban 30 Penetapan Serentak ke Pembimbing Kuota 3 (Tiga Kali Percobaan)

Alat uji beban tools/loadtest_supervisors/main.go dijalankan dengan melepaskan 30 goroutine serentak yang menetapkan 30 mahasiswa berbeda (std_001 s/d std_030) ke Dosen Pembimbing ID 1 (kuota 3). Berikut keluaran terminal asli dari 3 kali pengujian berturut-turut:

```
=== PERCOBAAN 1 ===
30 mahasiswa serentak ditetapkan ke pembimbing ID 1 (kuota 3)...
hasil request:
  201 OK                    3
  409 SUPERVISOR_FULL       27
waktu total : 32.414ms
Supervisor ID 1 (Dr. Budi Prasetyo, M.Kom) kuota=3 supervised_count=3 baris supervisions=3
-> KONSISTEN

=== PERCOBAAN 2 ===
30 mahasiswa serentak ditetapkan ke pembimbing ID 1 (kuota 3)...
hasil request:
  201 OK                    3
  409 SUPERVISOR_FULL       27
waktu total : 15.2474ms
Supervisor ID 1 (Dr. Budi Prasetyo, M.Kom) kuota=3 supervised_count=3 baris supervisions=3
-> KONSISTEN

=== PERCOBAAN 3 ===
30 mahasiswa serentak ditetapkan ke pembimbing ID 1 (kuota 3)...
hasil request:
  201 OK                    3
  409 SUPERVISOR_FULL       27
waktu total : 22.2924ms
Supervisor ID 1 (Dr. Budi Prasetyo, M.Kom) kuota=3 supervised_count=3 baris supervisions=3
-> KONSISTEN
```

4.2 Verifikasi Langsung Basis Data (supervised_count vs COUNT(*) supervisions)

Berikut hasil query SQL verifikasi langsung ke PostgreSQL yang membuktikan bahwa kolom supervised_count persis sama dengan jumlah baris nyata di tabel supervisions:

```
SELECT s.id, s.name, s.quota, s.supervised_count, COUNT(sp.id) AS
count_baris_supervisions,
       CASE WHEN s.supervised_count = COUNT(sp.id) THEN 'KONSISTEN' ELSE 'TIDAK KONSISTEN'
END AS status
FROM supervisors s
LEFT JOIN supervisions sp ON sp.supervisor_id = s.id
GROUP BY s.id, s.name, s.quota, s.supervised_count
ORDER BY s.id;

 id |           name            | quota | supervised_count | count_baris_supervisions |
status
----+---------------------------+-------+------------------+--------------------------+---
--------
  1 | Dr. Budi Prasetyo, M.Kom  |     3 |                3 |                        3 |
KONSISTEN
```

```
  2 | Dr. Siti Aminah, M.T      |    10 |               0 |                        0 |
KONSISTEN
  3 | Prof. Bambang Suryo, Ph.D |    10 |               0 |                        0 |
KONSISTEN
(3 rows)
```

4.3 Percobaan Pembanding (Negative Testing: Syarat Kuota Dihapus)

Sesuai petunjuk D.3, syarat kuota AND supervised_count < quota dihapus sementara dari method IncrementSupervised. Ketika diuji dengan 30 penetapan serentak yang sama, hasil yang diperoleh adalah:

```
30 mahasiswa serentak ditetapkan ke pembimbing ID 1 (kuota 3)...
hasil request:
  201 OK                    3
  500 INTERNAL_ERROR        27
waktu total : 237.7565ms
Supervisor ID 1 (Dr. Budi Prasetyo, M.Kom) kuota=3 supervised_count=3 baris supervisions=3
-> KONSISTEN
```

Analisis Percobaan Pembanding:

Saat klausul WHERE aplikasi dihilangkan, aplikasi mencoba menaikkan nilai supervised_count melebihi kuota 3. Hal ini memicu jaring pengaman database yaitu constraint CHECK (supervised_count <= quota), sehingga PostgreSQL menolak transaksi dengan error kode 500. Percobaan ini membuktikan bahwa alat uji mampu mendeteksi ketiadaan kontrol kuota aplikasi, dan membuktikan pentingnya UPDATE bersyarat di level kode Go agar menghasilkan respon 409 yang rapi.

BAB V — D.4 ANALISIS SINGKAT

1. Kuota pembimbing cukup dijaga UPDATE bersyarat, tetapi aturan 'satu mahasiswa satu pembimbing' saat terjadi pergantian membutuhkan kunci tambahan. Jelaskan mengapa, dengan menunjuk baris kode Anda.

UPDATE bersyarat yang atomik (WHERE supervised_count < quota) bekerja secara mandiri menjaga kapasitas pembimbing karena hanya memodifikasi satu baris pada tabel supervisors. Namun, aturan 'satu mahasiswa hanya boleh memiliki satu pembimbing' saat pergantian (Switch) melibatkan status relasional antar-tabel (students, supervisors, dan supervisions). Jika terdapat dua request pergantian pembimbing serentak untuk mahasiswa yang sama (misalnya request 1 ingin pindah ke Dosen B dan request 2 ke Dosen C), kedua request dapat secara bersamaan membaca pembimbing lama mahasiswa (Dosen A), lalu keduanya sama-sama mengurangi kuota Dosen A dan menambahkan kuota ke Dosen B dan C secara berlebih. Oleh karena itu, dibutuhkan penguncian pesimistik tambahan pada baris mahasiswa melalui pemanggilan repo.LockStudent(ctx, req.StudentID) (app/service/supervisor_service.go baris 112) yang mengeksekusi SELECT id FROM students WHERE id = $1 FOR UPDATE. Kunci ini memastikan bahwa request kedua wajib mengantre sampai transaksi pergantian pertama selesai di-commit atau di-rollback.

2. Seorang admin membuka form ubah kuota pembimbing, lalu meninggalkannya 30 menit sebelum menekan Simpan. Apa yang terjadi bila Anda memakai pessimistic locking untuk endpoint itu? Apa yang terjadi dengan optimistic locking?

Jika menggunakan pessimistic locking (SELECT ... FOR UPDATE), koneksi database dari pool akan dibuka dan baris pembimbing tersebut terkunci eksklusif selama 30 menit penuh. Hal ini menyandera 1 dari 10 koneksi pool aplikasi, dan seluruh mahasiswa yang mencoba mendaftar atau berpindah ke pembimbing tersebut selama 30 menit akan mengalami macet total (blocking) hingga

gagal akibat context deadline exceeded (5 detik). Sebaliknya, dengan optimistic locking (kolom version), basis data sama sekali tidak dikunci selama admin membuka form. Transaksi mahasiswa lain tetap berjalan normal. Ketika admin menekan tombol Simpan setelah 30 menit, sistem mengeksekusi UPDATE bersyarat yang memeriksa nomor versi (app/repository/supervisor_repository.go baris 186). Jika selama 30 menit data kuota telah diubah oleh admin lain, request akan ditolak dengan error 409 VERSION_CONFLICT secara aman tanpa pernah menyandera sumber daya server.

3. WithTx dapat menjalankan fn lebih dari sekali. Seandainya di dalam fn Anda mengirim email pemberitahuan ke dosen pembimbing, apa saja yang dapat salah? Di mana seharusnya email itu dikirim?

Fungsi TxManager.WithTx memiliki mekanisme retry otomatis bila terjadi serialization failure (40001) atau deadlock (40P01). Jika pemanggilan pengiriman email ditempatkan di dalam fungsi fn, maka: (1) Dosen pembimbing dapat menerima email pemberitahuan ganda (duplikat) sebanyak jumlah retry yang dijalankan sebelum transaksi berhasil. (2) Yang lebih fatal, pengiriman email adalah aksi eksternal jaringan yang tidak dapat di-rollback oleh basis data. Jika seluruh percobaan transaksi akhirnya gagal dan di-ROLLBACK, email sudah terlanjur terkirim ke dosen mengabarkan ada mahasiswa bimbingan baru, padahal di database data tersebut tidak pernah tersimpan. Oleh sebab itu, pengiriman email wajib diletakkan di luar blok transaksi, tepatnya pada layer service setelah pemanggilan s.tx.WithTx selesai dieksekusi dan dipastikan mengembalikan nilai nil (sukses di-COMMIT).

LAMPIRAN — BUKTI PENGUJIAN LENGKAP BAGIAN C

1. Uji Beban Rebutan Kelas IF399 (mode crowd — 60 Mahasiswa ke Kuota 5):

```
60 mahasiswa berebut kelas IF399 secara serentak...
hasil request:
  201 OK                    5
  409 COURSE_FULL           55
waktu total : 34.7398ms
IF399 kuota=5  enrolled_count=5  baris enrollments=5  -> KONSISTEN
```

2. Uji Beban Batas SKS & Rincian per Mahasiswa (mode credits — 10 Mahasiswa ke 12 Kelas 3 SKS):

```
10 mahasiswa, masing-masing mendaftar ke 12 kelas 3 SKS secara serentak (batas 24 SKS)...
hasil request:
  201 OK                    80
  409 CREDIT_LIMIT_EXCEEDED 50
waktu total : 79.2161ms
seluruh 10 mahasiswa <= 24 SKS -> KONSISTEN

Kueri Verifikasi Per Mahasiswa:
 username | total_kelas | total_sks
----------+-------------+-----------
 mhs001   |           8 |        24
 mhs002   |           8 |        24
 mhs003   |           8 |        24
 mhs004   |           8 |        24
 mhs005   |           8 |        24
 mhs006   |           8 |        24
 mhs007   |           8 |        24
 mhs008   |           8 |        24
```

```
 mhs009   |           8 |        24
 mhs010   |           8 |        24
(10 rows)
```

3. Uji Beban Pindah Kelas Serentak Bebas Deadlock (mode swap — 20 Pasang Tukar Kelas):

```
20 pasang mahasiswa pindah kelas berlawanan arah secara serentak...
hasil request:
  200 OK                    40
waktu total : 75.9501ms
deadlock terdeteksi PostgreSQL selama uji: 0
IF101 kuota=40 enrolled_count=20 baris enrollments=20 -> KONSISTEN
IF102 kuota=40 enrolled_count=20 baris enrollments=20 -> KONSISTEN
```

4. Uji Fungsional C.4 (Satu per Satu — 11 Skenario):

```
1. Daftar kelas kosong: Status=201
2. Daftar kelas ganda: Status=409 Code=ALREADY_ENROLLED
3. Drop kelas belum diambil: Status=409 Code=NOT_ENROLLED
4. Swap ID sama: Status=422 Code=VALIDATION_ERROR
5. Swap kelas sah: Status=200
6. Mahasiswa ubah kuota: Status=403
7. Admin ubah kuota valid: Status=200 (version naik dari 1 menjadi 2)
8. Admin version lama: Status=409 Code=VERSION_CONFLICT
9. Admin kuota di bawah peserta: Status=409 Code=QUOTA_BELOW_ENROLLED
```

5. Uji Ketahanan Server C.5 (175 Kegagalan Beruntun & pg_stat_activity):

```
GET /api/v1/health -> 200 OK: {"message":"server berjalan","status":"ok"}

SELECT pid, state, now() - xact_start AS lama, left(query, 60) AS query
FROM pg_stat_activity WHERE datname = 'praktikum_backend' AND state <> 'idle';

  pid  | state  |   lama   |                            query
-------+--------+----------+-----------------------------------------------------------------
 16776 | active | 00:00:00 | SELECT pid, state, now() - xact_start AS lama, left(query, 6
(1 row)
```